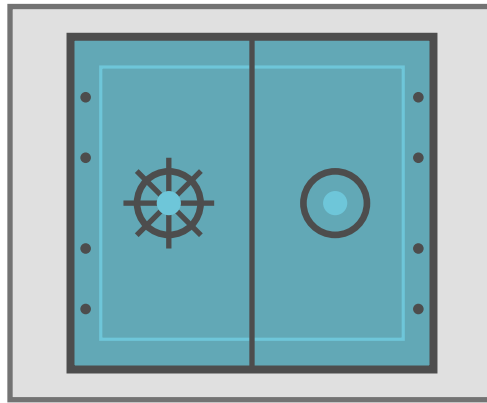


Criptografía y Seguridad

Criptografía

Cifrado Asimétrico y Firma Digital



Apunte integral — Teoría + Práctica

Clase 4 · Capítulos 9–12, *Introduction to Modern Cryptography* (Katz & Lindell)

Instituto Tecnológico de Buenos Aires (ITBA)

Documento elaborado a partir de las transcripciones de la clase teórica (Prof. Pablo Eduardo Abad) y de la clase práctica, junto con las diapositivas oficiales de ambas.

Índice

1. Introducción y contexto	3
2. El problema de la distribución de claves	3
2.1. ¿Cómo se comparten las claves?	3
2.2. El estallido cuadrático: por qué la clave privada no escala	4
2.3. KDC — Key Distribution Center (Trusted Third Party)	4
3. La revolución de la clave pública	5
3.1. La idea: problemas asimétricos	5
3.2. Llevarlo a criptosistemas: dos claves	5
4. Repaso de álgebra y aritmética modular	6
4.1. Estructuras algebraicas	6
4.2. Aritmética modular	7
5. Intercambio de claves Diffie–Hellman	7
5.1. Formalización del protocolo	7
5.2. Seguridad frente a ataques pasivos: el experimento $KE_{A,\Pi}$	7
5.3. El algoritmo Diffie–Hellman	8
5.4. La seguridad de DH: el problema del logaritmo discreto	9
5.5. Diffie–Hellman en la práctica: MiTM	9
6. Cifrado asimétrico (criptosistema de clave pública)	10
6.1. Modelo de seguridad: indistinguibilidad y CPA	10
7. RSA	11
7.1. “Textbook” RSA (versión determinística)	11
7.1.1. Por qué funciona (Euler / Fermat)	11
7.2. Ejemplo numérico RSA (teoría, parámetros ilustrativos pequeños)	12
7.3. PKCS #1 v1.5: hacer RSA no determinístico	12
7.4. Tamaño de claves	12
8. ElGamal (cifrado basado en DH)	12
8.1. Diferencias con RSA	13
8.2. Ejemplo numérico ElGamal (teoría, parámetros pequeños)	13
8.3. El nivel de seguridad en números	13
9. Cifrado híbrido	13
10. Firma digital	14
10.1. Firma digital vs. MAC	15
10.2. Seguridad: el experimento $Sig\text{-}forge_{A,\Pi}$	15
10.3. Textbook RSA-Signature (inseguro)	15
10.4. Hashed RSA: se firma el hash	16
10.5. Digital Signature Standard (DSS / DSA)	16
11. PKI: Infraestructura de Clave Pública	17
12. Práctica con OpenSSL y aplicación: TLS	17
13. Cross-check: qué agrega la práctica	19

Lectura recomendada

19

1 Introducción y contexto

En las clases anteriores se estudió la criptografía como un cuerpo de construcciones que resuelven problemas de seguridad con garantías formales. Se vieron los **criptosistemas** (garantía de no extraer información $\Rightarrow$ **confidencialidad**) y los **MAC** y **funciones de hash** (resistencia a falsificaciones / preimágenes $\Rightarrow$ **integridad**). Todas esas construcciones tienen algo en común: son **simétricas**, es decir, dependen de una *única clave secreta* compartida.

A alto nivel, los servicios de seguridad son tres: **Confidencialidad, Integridad y Disponibilidad**. La criptografía cubre los dos primeros; la disponibilidad *no* se resuelve con mecanismos criptográficos (se ve en la segunda parte de la materia).

Esta clase **revisita** esos mismos servicios desde otro modelo mental: el **modelo asimétrico** o de **clave pública**. Veremos que hay problemas muy difíciles de resolver con criptografía simétrica que se vuelven triviales con las construcciones nuevas. El recorrido es:

- **Intercambio de claves** (Diffie–Hellman): *algo totalmente nuevo*, que no se podía hacer con lo anterior.
- **Cifrado asimétrico** (RSA, ElGamal): análogo a los criptosistemas.
- **Firma digital** (RSA-Signature, DSS): análogo a los MAC.

2 El problema de la distribución de claves

La forma de entrar al tema es volver a un problema que se venía *ignorando*: el **problema de la distribución de claves**. Combinando MAC y criptosistemas se puede armar un **cifrado autenticado** (CCA-Secure), que garantiza confidencialidad e integridad de un mensaje. Pero esa propiedad **no es gratuita**.

La premisa oculta

Para que el cifrado autenticado funcione, **emisor y receptor deben conocer de antemano una misma clave**. Toda la seguridad descansa en que *un atacante nunca tenga acceso a la clave*. Consecuencia práctica directa:

Las claves NO pueden transmitirse por un canal inseguro.

Se puede transmitir *información cifrada* por un canal inseguro sin problema; la **clave**, no.

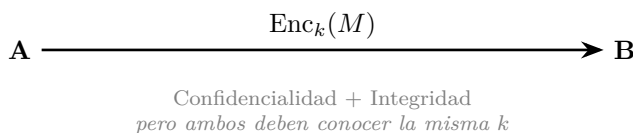


Figura 1: Un criptosistema CCA-Secure transmite con confidencialidad e integridad, pero **requiere clave compartida**. (*Recreación de la diapositiva.*)

2.1 ¿Cómo se comparten las claves?

- **Dos puntos (punto a punto)**: usar un **canal alternativo seguro**. Puede ser desde dos personas que se encuentran físicamente y comparten la clave, hasta un canal caro pero seguro reservado solo para transmitir la clave. Una variante (vista en la 2.^a parte) es la *seguridad por capas*: usar un canal alternativo *independiente*, de modo que comprometer ambos canales a la vez sea cuadráticamente más difícil.

- **Múltiples puntos:** el escenario se complica. Aparecen dos enfoques: *una clave por cada combinación* o *un punto único de confianza* (TTP).

2.2 El estallido cuadrático: por qué la clave privada no escala

Si cualquier par de entidades puede comunicarse, hace falta **una clave por cada par** posible. Con n participantes:

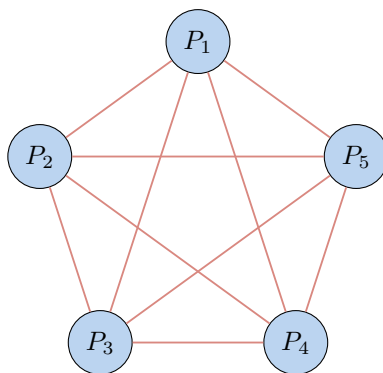
$$\#claves = \binom{n}{2} = \frac{n(n-1)}{2} = O(n^2), \quad \text{cada parte administra } n-1 \text{ claves.}$$

Ejemplo numérico (clase teórica): la empresa de 100 000 empleados

Si 100 000 empleados deben poder comunicarse de a pares:

$\#claves \approx 100\,000^2 = 10^{10}$ claves, cada empleado debe conocer 99 999 claves ajenas+la suya.

Es inmanejable de escalar. Esta explosión n^2 es *exactamente* la motivación para buscar otro paradigma.



Con $n = 5$: $\binom{5}{2} = 10$ claves.
Con $n = 100\,000$: $\sim 10^{10}$ claves.

Figura 2: Distribución de claves en criptografía simétrica: cada arista es una clave distinta. El crecimiento $O(n^2)$ es la limitación que **motiva la clave pública**. (*Recreación / integración teoría-práctica.*)

2.3 KDC — Key Distribution Center (Trusted Third Party)

La solución clásica al caso multipunto es un **tercero de confianza** (*Trusted Third Party*), llamado en la literatura **KDC**.

KDC — Centro de Distribución de Claves

- Es un **punto único** que centraliza el intercambio. Cada entidad comparte *una sola* clave con el KDC: k_A para A , k_B para B , etc.
- Si A quiere comunicarse con B : $A \rightarrow \text{KDC} : B$. El KDC genera una **clave de sesión** s y la envía cifrada a cada parte: $\text{KDC} \rightarrow A : \{s\}k_A$ y $\text{KDC} \rightarrow B : \{s\}k_B$.
- Luego A y B se comunican entre sí cifrando con s : $\{\dots\}s$.

Resultado: solo n claves para n entidades (cada una guarda únicamente su k_u).

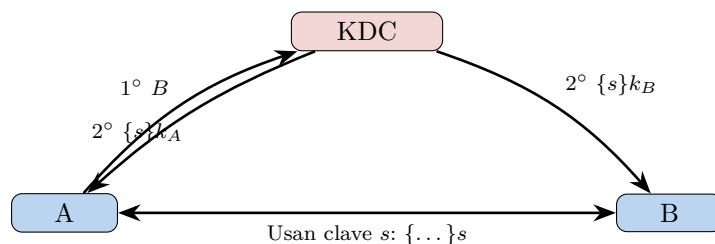


Figura 3: Protocolo KDC. Ejemplo real: **Needham–Schroeder** (base de *Kerberos* y de *Active Directory*). (Recreación de la diapositiva práctica.)

Pros y contras del KDC (clase práctica)

- + Cada uno guarda solo k_u ; solo un sitio de riesgo: el KDC.
- **Centralización:** si hay un *ataque al KDC* o una *falla en el KDC*, **se cae todo**. Es el **único punto de confianza / único punto de falla**.

Ejemplos de tecnologías basadas en este principio: **Kerberos** (estándar en mundos Unix) y **Active Directory** (estándar Microsoft). El protocolo *Needham–Schroeder* añade defensas contra ataques de *replay* y usa *timestamps*; se ve en detalle en la Clase 5.

Limitaciones de la criptografía de clave privada (resumen de la práctica): (1) **Distribución de clave**, (2) **Almacenamiento de clave**, (3) **Sistemas abiertos** (*open systems*). El KDC ataca (1) y (2); los *sistemas abiertos* —donde no se puede pre-compartir nada— son lo que finalmente empuja a la **Public Key Revolution**.

3 La revolución de la clave pública

Diffie & Hellman, 1976

“We stand today on the brink of a revolution in cryptography” — Whitfield **Diffie** y Martin **Hellman** (1976). Su ensayo es **seminal**: hay un antes y un después.

3.1 La idea: problemas asimétricos

La criptografía asimétrica se basa en la **asimetría de ciertos problemas**: funciones **fáciles en un sentido y difíciles de invertir** en el otro.

- **Analogía del candado:** cerrar un candado es facilísimo; abrirlo sin la llave es muy difícil (con la llave, igual de fácil que cerrarlo).
- **Problema SAT** (satisfactibilidad booleana): hallar una asignación que haga verdadera una proposición lógica es *difícil de resolver*, pero *verificar* una asignación dada es instantáneo.
- **Potencia vs. logaritmo:** la potencia es fácil de calcular; el *logaritmo discreto* (en aritmética entera/modular) es difícil.

3.2 Llevarlo a criptosistemas: dos claves

¿Y si en vez de una misma clave para cifrar y descifrar usamos **dos claves independientes**? Una para cifrar, otra para descifrar, de modo que *conocer una no permita derivar la otra*.

Claves asimétricas

- Aun si un atacante posee la clave de **cifrado**, *no* puede recuperar el mensaje. $\Rightarrow$ ¡se la puede **publicar a propósito**!
- Por eso se llama **criptografía de clave pública**: clave **pública** (cifra, emisor) y clave **privada** (descifra, receptor).
- La privada *está relacionada* con la pública (se podría obtener *en teoría*), pero **obtenerla no es una operación sencilla**: son dos valores distintos, vinculados pero computacionalmente separados.

Las tres primitivas que proponen Diffie y Hellman

1. **Encriptación** $\rightarrow$ la materializan luego **RSA / ElGamal**.
2. **MAC (firma digital)** $\rightarrow$ permite **no repudio**.
3. **Intercambio de claves** $\rightarrow$ lo presentan **completo**: es el algoritmo *Diffie-Hellman*.

En la firma, las claves se usan **al revés**: se **firma con la privada** y se **verifica con la pública**.

4 Repaso de álgebra y aritmética modular

El protocolo Diffie-Hellman requiere ponerse de acuerdo sobre un **grupo algebraico** y un **generador**.

4.1 Estructuras algebraicas

- **Grupo** $(G, +)$: conjunto y operación con *clausura, asociatividad, elemento neutro e inverso*.
- **Subgrupo** $(G, G', +)$: $(G, +)$ y $(G', +)$ son grupos, $G \supseteq G'$, $G' \neq \emptyset$.
- **Grupo abeliano**: grupo + *conmutatividad*.
- **Anillo** $(G, +, *)$: $(G, +)$ abeliano + clausura, asociatividad y distributividad de $*$ sobre $+$.
- **Cuerpo / campo** $(G, +, *)$: anillo + conmutatividad, neutro e inverso de $*$ (este último, en $G \setminus \{\text{neutro de } +\}$).

Grupos finitos cíclicos y generadores

$G = (\{g^n \mid n \in \mathbb{Z}\}, +)$ donde $\hat{}$ es la aplicación sucesiva de $+$.

- Todos los grupos de tamaño n son isomorfos. Grupo canónico $\mathbb{Z}_n = (\{0, 1, \dots, n-1\}, +)$, con $g^n = e$ (neutro).
- **Generador**: g tal que g es *primo relativo* a n . Hay $\Phi(n)$ generadores.
- $\text{ord}(g) = \#$ elementos del subgrupo cíclico generado por g ; g es **elemento primitivo** si $\text{ord}(g) = n$ (recorre *todos* los elementos).

Campo de Galois (campo finito): todo campo finito tiene tamaño p^n con p primo y n entero. Campo canónico $\mathbb{Z}_p^* = (\{k \mid k \in \mathbb{Z}_p \setminus \{0\}, \text{mcd}(k, p) = 1\}, +, *)$ con p primo, de tamaño $p-1$.

4.2 Aritmética modular

- $a \equiv b \pmod{n} \iff a - b = k \cdot n$. Se reduce al grupo canónico $0 \leq a < n$.
- Ejemplos: $2 + 8 \pmod{5} = 0$; $4 \cdot 3 \pmod{7} = 5$.
- Si n es **primo** $\Rightarrow$ campo finito $\mathbb{Z}_p$. Si n **no** es primo $\Rightarrow$ anillo; tomando solo los k con $\text{mcd}(k, n) = 1$ queda el **grupo multiplicativo** $(\mathbb{Z}_n^*, *)$.

Euler y Fermat — la maquinaria de RSA

- Si $\text{mcd}(k, n) = 1 \Rightarrow \exists k^{-1}$ tal que $k \cdot k^{-1} \equiv 1 \pmod{n}$.
- **Teorema de Euler:** $a^{\Phi(n)} \equiv 1 \pmod{n}$, donde $\Phi(n)$ es el tamaño de $\mathbb{Z}_n^*$.
- **Pequeño teorema de Fermat** (caso p primo): $a^{p-1} \equiv 1 \pmod{p}$. (*Fermat para primos; Euler lo extendió a cualquier n .*)
- Cálculo de Φ : $\Phi(n \cdot m) = \Phi(n)\Phi(m)$ si $\text{mcd}(n, m) = 1$; $\Phi(p^k) = p^k - p^{k-1} = p^{k-1}(p-1)$ si p primo. En particular $\Phi(p) = p-1$.

5 Intercambio de claves Diffie–Hellman

5.1 Formalización del protocolo

Un protocolo de intercambio de claves $\Pi(n)$ ejecutado por dos partes produce:

$$\Pi : (n) \rightarrow (\text{Trans}, k_a, k_b)$$

No tiene entrada (salvo el parámetro de seguridad n). Su salida es un conjunto de **mensajes intercambiados** (transcripción), una clave k_a conocida solo por una parte y una k_b conocida solo por la otra.

Condición fundamental

$$k_a = k_b$$

Tras intercambiar cierta información, ambos lados deben llegar a la **misma clave**, cada uno calculándola *solo con la información que posee en su extremo*, para usarla luego en otras construcciones (p. ej. cifrado simétrico).

5.2 Seguridad frente a ataques pasivos: el experimento $KE_{A,\Pi}$

Dado un adversario A y una primitiva de intercambio de claves Π :

1. Se ejecuta Π ; sea $k = k_a = k_b$.
2. Se genera $b \leftarrow \{0, 1\}$.
3. Si $b = 0 \Rightarrow k' \leftarrow \{0, 1\}^n$ (aleatoria); si $b = 1 \Rightarrow k' = k$.
4. A obtiene Trans y k' , y emite $b' \in \{0, 1\}$.

$KE_{A,\Pi} = 1$ si $b = b'$ (A gana). El protocolo es seguro si $\Pr[KE_{A,\Pi} = 1] < \frac{1}{2} + \varepsilon$.

No se le pide al atacante *adivinar* la clave, sino algo **más fácil**: decidir si la clave que le dan es la del intercambio o una aleatoria. Así garantizamos que no extraiga *ninguna* información parcial (p. ej. “la clave termina en 001”, que reduciría la búsqueda $8\times$). Si A no puede sesgar su respuesta, gana en promedio la mitad de las pruebas (0,5); si llega al 60 %, *encontró algo* y el protocolo falla.

5.3 El algoritmo Diffie–Hellman

Tres parámetros **públicos**: un grupo G , su tamaño/orden q y un generador g . $\mathbb{Z}_q = \{0, 1, \dots, q-1\}$.

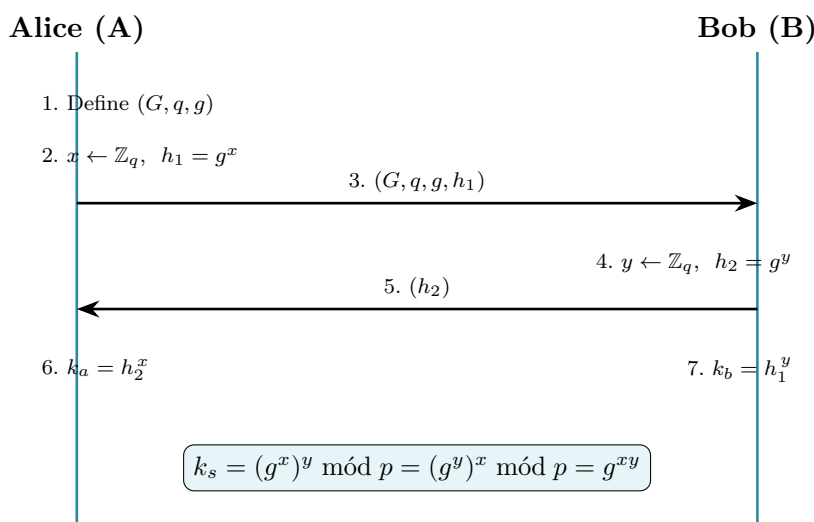


Figura 4: Intercambio Diffie–Hellman. En azul, lo que viaja por el canal (a lo que accede el atacante): (G, q, g, h_1) y h_2 . Si A y B se conocen de antemano, pueden tener predefinidos (G, q, g) . (*Recreación de la diapositiva.*)

Aclaración del profe: DH *acuerda* la clave, no la *transporta*

La clave acordada g^{xy} **nunca viaja** por el canal. Cada parte la calcula en su extremo: A eleva lo que recibió (h_2) a su secreto x ; B eleva lo que recibió (h_1) a su secreto y . Ambos obtienen g^{xy} *sin haberlo enviado jamás*.

Ejemplo numérico completo (clase práctica): $\mathbb{Z}_7, g = 3$

Grupo $\mathbb{Z}_7$ con generador $g = 3$. Como 3 es generador, sus potencias barren todo $\{1, \dots, 6\}$ antes de repetirse:

$$\begin{aligned} 3^0 &\equiv 1, & 3^1 &\equiv 3, & 3^2 &= 9 \equiv 2, \\ 3^3 &= 27 \equiv 6, & 3^4 &= 81 \equiv 4, & 3^5 &= 243 \equiv 5, & (3^6 &\equiv 1) \quad (\text{mód } 7). \end{aligned}$$

Intercambio:

- A piensa $x = 3$ y calcula $h_1 = 3^3 \equiv 6 \pmod{7}$; envía 6 (no envía 3).
- B piensa $y = 4$ y calcula $h_2 = 3^4 \equiv 4 \pmod{7}$; envía 4.
- A : $k_a = h_2^x = 4^3 \equiv 1 \pmod{7}$.
- B : $k_b = h_1^y = 6^4 \equiv 1 \pmod{7}$.

Coinciden: $k_a = k_b = 1 = 3^{3 \cdot 4} = 3^{12}$. El valor 3^{12} *nunca viajó*, pero ambos lo acuerdan. (*Aquí $4^3 = 3^{4 \cdot 3}$ y $6^4 = 3^{3 \cdot 4}$: la misma cuenta por ambos caminos.*)

5.4 La seguridad de DH: el problema del logaritmo discreto

Logaritmo discreto y conjetura de decisión DH

Dados g^x y g^y , no debería ser posible obtener x o y : ese es el **problema del logaritmo discreto**, sin solución eficiente. En el ejemplo de $\mathbb{Z}_7$ es trivial (hay pocas opciones); con un **primo grande** se vuelve intratable.

- El logaritmo discreto es **condición necesaria pero no suficiente**.
- **Conjetura de decisión DH (DDH)**: dados g, g^x, g^y , un adversario no puede distinguir g^{xy} de un valor aleatorio.
- *Nota*: la formulación de la conjetura se hizo **muchos años después** de publicarse el algoritmo, como “parche” para darle marco teórico sólido. Hoy se sabe que el problema está en una categoría tipo NP-Hard.

Computación cuántica

El problema del logaritmo discreto pertenece a la categoría que resuelve el **algoritmo de Shor** en una computadora cuántica. La computación cuántica afecta a **toda la parte asimétrica** de la criptografía. La parte *simétrica* es esencialmente inmune; ya hay **criptografía post-cuántica** implementándose en las librerías importantes, aunque sigue siendo exploratoria (problemas de coherencia, etc.).

5.5 Diffie–Hellman en la práctica: MiTM

La versión original requiere canal autenticado

DH protege contra atacantes **pasivos**, pero la versión original **requiere un canal de transmisión autenticado**: un atacante que pueda *modificar* mensajes rompe la seguridad. Ninguna parte verifica *con quién* está intercambiando. Se complementa con **MAC o firmas digitales**.

Ataque *Man-in-the-Middle* sobre DH (clase práctica)

El gran problema que DH no contempla es la **autenticación**. La “bruja malvada” M se mete en el medio: A cree hablar con B , pero ejecuta el protocolo completo *con* M , y M a su vez lo ejecuta con B . A y M acuerdan una clave; M y B acuerdan otra. Para A y B todo es *indetectable*.

- **Man-in-the-Middle (MIM)**: el adversario está en medio de los dos extremos y engaña a *ambos* (intercepta y modifica). Puede ser *activo* o *pasivo* (registrar para un ataque futuro).
- **Masquerading / Impersonating**: solo *una* parte ejecuta el protocolo y el adversario se hace pasar por la otra. La víctima suplantada *ni se entera*.
- **Replay** (repetición de mensajes) y **Key Reuse** (reuso de claves): se mitigan con **timestamps** (marcas de tiempo que prueban que el mensaje es reciente) y con rotación/actualización de claves.

6 Cifrado asimétrico (criptosistema de clave pública)

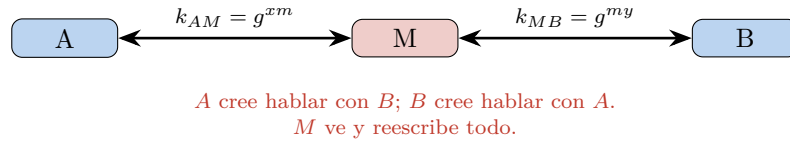


Figura 5: MiTM sobre DH: sin autenticación, M acuerda una clave con cada lado. (*Recreación / integración con la práctica.*)

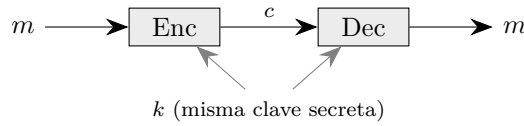
Criptosistema asimétrico — terna de algoritmos

$\text{Gen} : () \rightarrow \mathcal{PK} \times \mathcal{SK}$	Generador de clave
$\text{Enc} : \mathcal{PK} \times \mathcal{P} \rightarrow \mathcal{C}$	Cifrado
$\text{Dec} : \mathcal{SK} \times \mathcal{C} \rightarrow \mathcal{P}$	Descifrado

Propiedad básica: para todo m y todo par (pk, sk) válido, $d_{sk}(e_{pk}(m)) = m$.

Conjuntos: $\mathcal{PK}$ = claves públicas; $\mathcal{SK}$ = claves privadas; $\mathcal{P}$ = mensajes (texto plano); $\mathcal{C}$ = mensajes cifrados.

Simétrico



Asimétrico

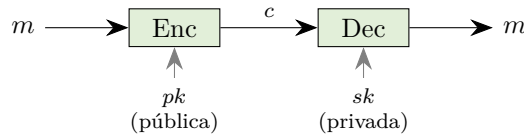


Figura 6: Simétrico (una clave para cifrar y descifrar) vs. asimétrico (clave pública cifra, privada descifra). (*Recreación / integración.*)

6.1 Modelo de seguridad: indistinguibilidad y CPA

Test de indistinguibilidad frente a eavesdropping $Eav_{A,\Pi}$:

1. Se genera $(pk, sk) \leftarrow K$.
2. A **obtiene** pk y genera (m_0, m_1) .
3. Se genera $b \leftarrow \{0, 1\}$.
4. A recibe $c = e_{pk}(m_b)$.
5. A emite $b' \in \{0, 1\}$.

$Eav_{A,\Pi} = 1$ si $b = b'$. Es indistinguible si $\Pr[Eav_{A,\Pi} = 1] < \frac{1}{2} + \varepsilon$.

La diferencia con el mundo simétrico: aquí al adversario **se le entrega la clave pública**. Por eso puede cifrar *cualquier* mensaje (tiene el oráculo de cifrado a disposición).

Consecuencia: CPA-Secure exige cifrado NO determinístico

Si Π es indistinguible para un adversario pasivo $\Rightarrow \Pi$ es **CPA-Secure**. Pero **CPA-Secure REQUIERE cifrado no determinístico**. Razonamiento: como el atacante conoce pk , si el cifrado fuera determinístico podría calcular él mismo $e_{pk}(m_0)$ y $e_{pk}(m_1)$ y comparar con c , ganando siempre. *No existen criptosistemas asimétricos seguros que sean determinísticos: hay que incluir no determinismo desde el vamos.*

7 RSA

7.1 “Textbook” RSA (versión determinística)

GenRSA — generación de claves

1. Elegir p, q primos. Calcular $n = p \cdot q$ (la “base”).
2. $\Phi(n) = (p - 1)(q - 1)$.
3. Elegir $e \leftarrow (0, \Phi(n))$ tal que $\text{mcd}(e, \Phi(n)) = 1$.
4. Calcular d tal que $e \cdot d \equiv 1 \pmod{\Phi(n)}$ (inverso modular, por Euclides extendido).
5. $pk = \langle n, e \rangle$, $sk = \langle n, d \rangle$.

$$\text{Enc}_{pk}(m) \equiv m^e \pmod{n}, \quad \text{Dec}_{sk}(c) = c^d \pmod{n}.$$

7.1.1 Por qué funciona (Euler / Fermat)

Como $d \equiv e^{-1} \pmod{\Phi(n)}$:

$$\begin{aligned} c^d \pmod{n} &= (m^e)^{e^{-1} \pmod{\Phi(n)}} \pmod{n} = m^{e \cdot e^{-1}} \pmod{n} = m^{1+k\Phi(n)} \pmod{n} \\ &= m \cdot (m^{\Phi(n)})^k \pmod{n} \equiv m^1 = m, \end{aligned}$$

usando $m^{\Phi(n)} \equiv 1 \pmod{n}$ (Euler). Cifrado y descifrado son **inversas** con el par de claves correcto.

Problemas del Textbook RSA

- **Es determinístico** $\Rightarrow$ no es CPA-Secure (mismo m y mismo $pk \Rightarrow$ siempre el mismo c ; el atacante prueba mensajes hasta dar con el cifrado).
- **e y m pequeños**: si $m^e < n$, no hay reducción modular y se puede calcular el logaritmo entero (recuperando m). Durante mucho tiempo se usó $e = 3$ para ahorrar tiempo $\Rightarrow$ riesgoso.
- **Módulos repetidos**: si dos pares de claves comparten el mismo n , es posible recuperar n y luego la clave privada.
- **Selección de primos**: no sirven dos primos cualesquiera; deben ser *primos fuertes* con ciertas propiedades (hay ataques).

7.2 Ejemplo numérico RSA (teoría, parámetros ilustrativos pequeños)

$p = 2357$, $q = 2551$, $n = p \cdot q = 6012707$,
 $\Phi(n) = (p-1)(q-1) = 6007800$, $e = 3674911$ (al azar), $d = 422191$ (Euclides ext.).
Cifrado de $m = 5234673$: $e(m) = 5234673^{3674911} \bmod 6012707 = \mathbf{3650502}$.
Descifrado de $m' = 3650502$: $d(m') = 3650502^{422191} \bmod 6012707 = \mathbf{5234673}$. ✓

7.3 PKCS #1 v1.5: hacer RSA no determinístico

Padding aleatorio (RSA Labs Public Key Cryptography Standard)

Sea k la longitud de n en bytes. Solo se permiten cifrar mensajes de hasta $k - 11$ bytes; sea D la longitud de m . Se construye:

$$m' = 00000000 \parallel 00000010 \parallel r \parallel 00000000 \parallel m$$

donde r son $k - D - 3$ bytes aleatorios distintos de 0 (la condición $\neq 0$ evita ambigüedades).

El padding aporta: (1) **no determinismo** (los bytes r aleatorios); (2) garantía de que el número sea > 0 ; (3) prefijo que evita problemas algebraicos. **Se cree que es CPA-Secure**, pero se hallaron ataques que muestran que *no* es CCA-Secure. Para usar RSA correctamente *siempre* hay que aplicar la normativa PKCS (padding + valores aleatorios).

7.4 Tamaño de claves

Para RSA, $n = p \cdot q$. En campos numéricos se recomienda $n \geq 1024$ bits; **hoy se usan 1536 o 2048 bits** (*rsa-2048*). El nivel de seguridad es relativo al tamaño de los conjuntos involucrados.

8 ElGamal (cifrado basado en DH)

ElGamal — terna de algoritmos

Generación: seleccionar G, q, g (campo G de tamaño q); $x \leftarrow \mathbb{Z}_q$, $h = g^x$.

$$pk = (G, q, g, h), \quad sk = (G, q, g, x).$$

Cifrado $e_{pk}(m)$: elegir $y \leftarrow \mathbb{Z}_q$; salida $c = (c_1, c_2) = (g^y, h^y \cdot m)$.

Descifrado: $d_{sk}(c) = c_2 / c_1^x$.

Conceptualmente es “un Diffie–Hellman convertido en criptosistema”: $h^y = g^{xy}$ hace de clave de sesión que enmascara m , y $c_1 = g^y$ permite al receptor reconstruir g^{xy} con su x .

Resultado teórico: si la prueba de decisión DH (DDH) es difícil en G , entonces ElGamal es CPA-Secure.

8.1 Diferencias con RSA

- ElGamal es **probabilístico** por naturaleza (el y aleatorio en cada cifrado $\Rightarrow$ no determinístico desde el vamos).
- Permite **reutilizar** los parámetros (G, q, g) .
- **No está limitado a campos numéricos**: admite anillos de polinomios (2^n elementos $\Rightarrow$ fácil mapear mensajes) y **curvas elípticas** (donde el problema de decisión DH es más complejo).

Nota del profe: ElGamal vs. RSA en la práctica

ElGamal es el **segundo criptosistema de clave pública más importante** y tiene más de 20 años. Si en una herramienta moderna aparece la sigla con “EC” (*Elliptic Curve*), casi seguro están trabajando con **ElGamal sobre curvas elípticas**, no con RSA. Sobre curvas elípticas el equivalente a una clave RSA de 2048 bits son ~ 320 bits (la práctica menciona $n \geq 320$ bits para curvas).

8.2 Ejemplo numérico ElGamal (teoría, parámetros pequeños)

$$G = \mathbb{Z}_q^*, \quad q = 2\,357, \quad g = 2, \quad x = 1\,751 \Rightarrow g^x \bmod q = 2^{1\,751} \bmod 2\,357 = 1\,185.$$

Cifrado de $m = 2\,035$, eligiendo $y = 1\,520$:

$$e(m) = (2^{1\,520} \bmod 2\,357, \quad 2\,035 \cdot 1\,185^{1\,520} \bmod 2\,357) = (1\,430, \quad 697).$$

Descifrado de $m' = (1\,430, 697)$:

$$d(m') = 1\,430^{-1\,751} \cdot 697 \bmod 2\,357 = \mathbf{2\,035}. \quad \checkmark$$

8.3 El nivel de seguridad en números

El nivel de seguridad es relativo al tamaño de los conjuntos. En RSA $n = p \cdot q$; en ElGamal $n = q$. Para campos numéricos $n \geq 1024$ bits (hoy 1536/2048). La fortaleza descansa en que **factorizar n** (RSA) o calcular el **logaritmo discreto** (ElGamal) no tiene solución polinómica; pero existen algoritmos *subexponenciales* (mejores que la fuerza bruta), lo que obliga a claves grandes. Las **curvas elípticas** no admiten ataque subexponencial conocido $\Rightarrow$ permiten claves mucho más chicas (~ 256 –320 bits).

9 Cifrado híbrido

El cifrado asimétrico es **mucho más lento** (potencias modulares sobre números enormes; cifrar un PDF completo sería inviable). La solución es **combinar**: usar lo asimétrico *solo* para transportar una clave simétrica, y lo simétrico para el mensaje.

Esquema de cifrado híbrido: público-asim (π) + privado-sim (π')

1. Gen^{hy} : $(pk, sk) \leftarrow \text{Gen}(1^n)$.

2. Enc^{hy} : Elegir $K \leftarrow \{0, 1\}^n$. Calcular

$$c_1 \leftarrow \text{Enc}_{pk}(K) \text{ (cifrado asimétrico)}, \quad c_2 \leftarrow \text{Enc}'_K(m) \text{ (cifrado simétrico)}.$$

Emitir (c_1, c_2) .

3. Dec^{hy} : $K := \text{Dec}_{sk}(c_1)$; $m := \text{Dec}'_K(c_2)$.

CPA-Secure siempre que: π sea CPA-Secure y π' sea seguro ante *eavesdropping*.

Como la clave K es chica, no es costoso cifrarla asimétricamente; el grueso (el mensaje) viaja por el esquema simétrico, rápido. Además K se elige **aleatoria en cada comunicación**, lo que resuelve elegantemente la **distribución de claves**. Esta es la base de protocolos reales como TLS.

10 Firma digital

Las firmas digitales son los **MAC asimétricos**: su objetivo es la **integridad**, pero con ventajas adicionales que les dan su nombre.

Firma digital — terna de algoritmos

$\text{Gen} : () \rightarrow \mathcal{PK} \times \mathcal{SK}$

Generador de clave

$\text{Sign} : \mathcal{SK} \times \mathcal{P} \rightarrow \mathcal{S}$

Firma

$\text{Vrfy} : \mathcal{PK} \times \mathcal{P} \times \mathcal{S} \rightarrow \{0, 1\}$

Verificación

Propiedad básica: para todo m y (pk, sk) válido, $\text{Vrfy}_{pk}(m, \text{Sign}_{sk}(m)) = 1$.

Conjuntos: $\mathcal{PK}$, $\mathcal{SK}$ como antes; $\mathcal{P}$ = mensajes; $\mathcal{S}$ = firmas posibles.

Aclaración del profe: las claves van al revés

A diferencia del cifrado, aquí se **firma con la clave privada** y se **verifica con la pública**. Si se firmara con la pública, *cualquiera* podría firmar y no habría integridad. Como solo el dueño de sk puede firmar, pero *cualquiera* con pk puede verificar, se obtiene la propiedad clave: el **no repudio**.

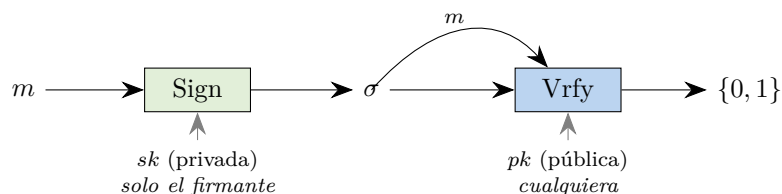


Figura 7: Firma digital: la **privada** firma, la **pública** verifica. (Recreación de la diapositiva.)

10.1 Firma digital vs. MAC

Ambos aseguran integridad	
Firma digital	MAC
Fácil distribución de clave	Una clave por cada receptor
Una sola firma la verifica <i>cualquier</i> receptor	Una MAC por cada clave
Verificable públicamente	<i>No</i> verificable públicamente
Provee no repudio	<i>No</i> provee no repudio
(más lento)	Más eficiente

No repudio (explicación del profe)

“No repudio” es un término más **legal** que matemático. Una firma de puño y letra es *no repudiable*: **válida hasta que se demuestre lo contrario**, y la carga de probar que “no fui yo” recae en el firmante. Con un **MAC** esto no se logra: para verificar hay que compartir la *misma* clave con la que se podría generar otra etiqueta $\Rightarrow$ cualquiera de las dos partes pudo haberla hecho. Con la **firma digital**, como se verifica con pk , la *única forma de haberla generado es teniendo sk* ; bajo la premisa de que sk nunca se comparte, “si se verifica con la clave pública de Pablo, es porque Pablo lo firmó”. En Argentina (pionera) la **ley de firma digital** le da el mismo valor legal que la firma manuscrita.

10.2 Seguridad: el experimento $Sig\text{-}forge_{A,\Pi}$

Dado un nivel de seguridad n , un adversario A y una firma digital $\Pi(n)$:

1. Se generan claves $(sk, pk) \leftarrow K$.
2. A obtiene $f(x) = \text{Sign}_{sk}(x)$ (oráculo de firma) y pk .
3. A realiza las evaluaciones que quiera de $f(x)$ (sea Q ese conjunto).
4. A emite (m, s) con $m \notin Q$.

$Sig\text{-}forge_{A,\Pi} = 1$ si $\text{Vrfy}_{pk}(m, s) = 1$ y $m \notin Q$ (falsificación).

Si no hay algoritmo atacante que gane esta prueba, la firma es **infalsificable**.

10.3 Textbook RSA-Signature (inseguro)

RSA-Signature — idéntico a RSA cifrado, pero invirtiendo las claves

Generación: igual que GenRSA, $pk = \langle n, e \rangle$, $sk = \langle n, d \rangle$.

$$\text{Sign}_{sk}(m) \equiv m^d \pmod{n}, \quad \text{Vrfy}_{pk}(m, s) : \text{¿ } m \stackrel{?}{=} s^e \pmod{n} ?$$

Este esquema, aunque común en la literatura, es INSEGURO

Ataque 1 — “de no mensaje”. El adversario:

$$s \leftarrow \mathcal{S} \text{ (firma al azar); } m = s^e \pmod{n}; \text{ emite } (m, s).$$

Como m se construyó a partir de s , por definición $s^e \equiv m$ y **pasa la verificación**, con un m que el atacante *nunca* consultó al oráculo. El mensaje no tiene sentido, pero el par (m, s) es válido $\Rightarrow$ falsificación.

Ataque 2 — maleabilidad multiplicativa. Dados $s_1 = f(m_1)$ y $s_2 = f(m_2)$:

$$\text{emite } (m_1 \cdot m_2, s_1 \cdot s_2).$$

Ejercicio (queda propuesto): verificar que pasa, pues $(s_1 s_2)^e = s_1^e s_2^e \equiv m_1 m_2 \pmod{n}$.

10.4 Hashed RSA: se firma el hash

Aclaración del profe: se firma el HASH, no el mensaje

Se introduce una **función de hash libre de colisiones** H :

$$\text{Sign}_{sk}(m) \equiv H(m)^d \pmod{n}, \quad \text{Vrfy}_{pk}(m, s) : \text{¿ } H(m) \stackrel{?}{=} s^e \pmod{n} ?$$

Por qué resuelve los ataques anteriores:

- **Resistencia a preimagen** $\Rightarrow$ no se puede invertir $H \Rightarrow$ el *ataque de no mensaje* no funciona (no se puede partir de s y hallar un m con $H(m) = s^e$).
- No se conoce a priori el valor del hash $\Rightarrow$ se frena el segundo ataque.
- **Bonus de eficiencia:** el hash tiene tamaño fijo, así que se firma algo *mucho más pequeño* sin importar el tamaño del documento.

Caveat: no tiene prueba de seguridad formal salvo asumiendo un **modelo ideal** de H (*random oracle*).

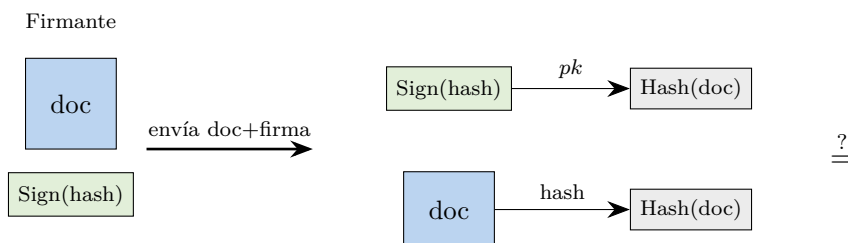


Figura 8: Verificación: de la firma se obtiene $H(\text{doc})$ con pk ; del documento se recalcula su hash. Si coinciden, la firma es válida. (*Recreación de la diapositiva práctica.*)

10.5 Digital Signature Standard (DSS / DSA)

DSS — generación de claves

- Seleccionar $H(x)$: SHA-1 o SHA-2.
- Tamaños (L, N) : $(1024, 160)$, $(2048, 224)$, $(2048, 256)$ o $(3072, 256)$.
- $q \leftarrow$ primo de N bits; $p \leftarrow$ primo de L bits con $(p-1) \equiv 0 \pmod{q}$.
- $g \leftarrow$ generador de orden q mód p , con $g^{(p-1)/q} \neq 1$. ($G = \mathbb{Z}_p^*$).
- $x \leftarrow \mathbb{Z}_q$, $y = g^x \pmod{p}$. $pk = (p, q, g, y)$, $sk = (p, q, g, x)$.

DSS — firma y verificación

Firma $\text{Sign}_{sk}(m)$: $k \leftarrow \mathbb{Z}_q$, $r = (g^k \bmod p) \bmod q$,

$$s = [(H(m) + x \cdot r) \cdot k^{-1}] \bmod q, \quad \text{Sign}_{sk}(m) = (r, s).$$

Verificación $\text{Vrfy}_{pk}(m, (r, s))$:

$$u_1 = [H(m) \cdot s^{-1}] \bmod q, \quad u_2 = [r \cdot s^{-1}] \bmod q,$$

$$r \stackrel{?}{=} (g^{u_1} \cdot y^{u_2} \bmod p) \bmod q$$

DSS está definido sobre cualquier campo, por lo que existe la firma sobre **curvas elípticas** (ECDSA). Si una sigla de firma incluye “EC”, es DSS sobre curvas.

11 PKI: Infraestructura de Clave Pública

La clave pública resuelve la distribución, pero introduce un problema nuevo: **¿cómo sé que una clave pública pertenece realmente a quien dice ser?** Si alguien me entrega una “clave pública de B ” que en realidad es de M , cifraré para M creyendo cifrar para B (mismo patrón que el MiTM). La respuesta son los **certificados** y una **cadena de confianza**.

PKI — Public Key Infrastructure

Define **cómo distribuir** $\langle pk, sk \rangle$ mediante **certificados**. Un certificado es un archivo que contiene:

- La **clave pública del titular** (la del que dice ser el emisor).
- **Quién firma** esa clave pública (la autoridad certificante).
- **Validez**: fechas de emisión/expiración y estado de **revocación**.

Se organiza en **jerarquías** con una **autoridad certificante raíz** en el tope, que certifica a las demás. Hay que *confiar* en la autoridad certificante (igual que antes se confiaba en el KDC: *siempre hace falta un nivel de confianza*).

Aclaración del profe: el certificado lleva la pública del titular

La **clave pública** es lo único que se entrega al receptor; es un archivo significativamente *más pequeño* que el de la clave privada (que además contiene p, q, n, e, d y valores auxiliares). El **certificado** es *otro* archivo: lleva la clave pública del titular, la firma de la autoridad, la fecha de validez y el estado de revocación. Si el emisor sufre un ataque, sus certificados se **revocan**, por lo que el estado debe mantenerse permanentemente.

12 Práctica con OpenSSL y aplicación: TLS

Generar un par de claves RSA con OpenSSL (Guía 4)

```
$ openssl genrsa -out privada.pem 2048
Generating RSA private key, 2048 bit long modulus
.....+++
.....+++
e is 65537 (0x10001)
```

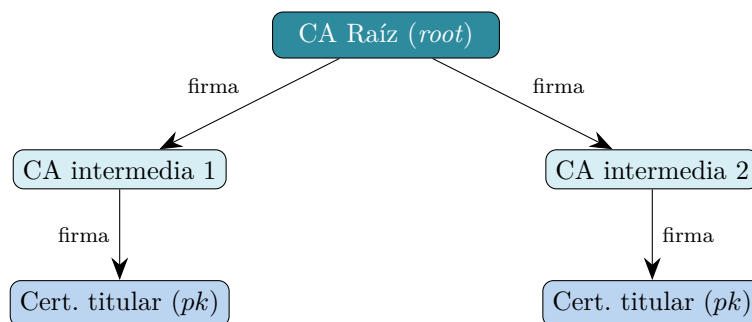


Figura 9: Cadena de certificados PKI: la raíz firma a las intermedias y estas a los titulares. La validez se verifica eslabón por eslabón hasta la raíz de confianza. Ej.: pki.jgm.gov.ar (firma digital de la Rep. Argentina). (*Recreación de la diapositiva práctica.*)

```
$ file privada.pem
privada.pem: PEM RSA private key
```

El archivo `privada.pem` contiene **ambas** claves: la privada y la pública, más p , q , n , los exponentes e y d y valores auxiliares para agilizar los cálculos. **Cuanto más grande la clave, más tarda en calcular los primos p y q** ; el exponente e suele ser fijo (65537) y se espera a obtener el resto aleatoriamente.

Inspeccionar y extraer la clave pública

```
# Ver la clave privada en texto (p, q, N, e, d, ...):
$ openssl rsa -in privada.pem -text -out privada.txt
writing RSA key

# Extraer SOLO la clave pública (la que se publica):
$ openssl rsa -in privada.pem -pubout -out publica.pem
```

El fichero con la clave **pública** es **mucho más pequeño**: solo contiene los datos de la pública ($\langle n, e \rangle$), nada de p , q , d . Esa es la que se da al receptor. Más info: flu-project.com.

TLS — Transport Layer Security

Proporciona **cifrado, autenticación e integridad** entre ambos extremos de la conexión (capa de transporte). Combina todas las piezas vistas:

- **Confidencialidad**: combina cifrado simétrico y asimétrico (esquema híbrido).
- **Autenticación**: autentica cliente y servidor (con certificados/PKI).
- **Integridad**: usa hash.

Al iniciar la conexión hay un *handshake* donde las partes se autentican y **acuerdan los algoritmos** a usar. Usa un **Diffie–Hellman autenticado** para el intercambio de claves (no el DH “sencillo”), cifrados simétricos modernos (AES-CCM, AES-GCM) y hash con *AEAD* (cifrado autenticado). RFCs: **RFC 2246** (primera versión) y **RFC 8446** (TLS 1.3).

13 Cross-check: qué agrega la práctica

Aportes de la clase práctica respecto de la teórica

- **Motivación de clave pública desde las limitaciones de la clave privada:** la práctica enumera explícitamente las tres limitaciones (distribución, almacenamiento, sistemas abiertos) y muestra cómo el KDC ataca las dos primeras y los *open systems* fuerzan la *Public Key Revolution*.
- **KDC en detalle:** protocolo $A \rightarrow \text{KDC} : B, \{s\}_{k_A}/\{s\}_{k_B}$, con el diagrama y el ejemplo *Needham-Schroeder* (que la teórica solo menciona como TTP).
- **Ejemplo DH numérico paso a paso** en $\mathbb{Z}_7$ con $g = 3$ (la teórica usa el grupo aditivo módulo 7 como repaso; la práctica lo resuelve completo con potencias).
- **Catálogo de ataques sobre DH:** Impersonating/Masquerading, MIM, Replay, Key Reuse, con definiciones precisas y la distinción MIM (engaña a los dos) vs. Masquerading (la víctima ni se entera).
- **Esquema de cifrado híbrido** formalizado (c_1 asimétrico para K , c_2 simétrico para m) — la teórica lo menciona como “combinar”, la práctica da el algoritmo y la condición de seguridad.
- **Tabla Firma Digital vs. MAC** lado a lado.
- **Diagrama de verificación de firma del hash** ($\text{doc} + \text{Sign}(\text{hash}) \rightarrow \text{comparar hashes}$).
- **PKI/certificados y TLS** como cierre aplicado, con el portal real `pki.jgm.gov.ar` y los RFC de TLS.
- **Práctica OpenSSL** (`genrsa`, `-text`, `-pubout`) que no está en la teórica.

La **teórica**, en cambio, aporta el rigor de los modelos de seguridad ($KE_{A,\Pi}$, $E_{A,\Pi}$, Sig-forge), la deducción de Euler/Fermat, el padding PKCS#1, ElGamal/curvas elípticas, DSS y el impacto cuántico.

Lectura recomendada

Capítulos 9–12

Introduction to Modern Cryptography
Katz & Lindell

Complementos de la práctica: *Applied Cryptography* — Bruce Schneier; **TLS**: RFC 2246 (primera versión) y RFC 8446 (TLS 1.3).